

API REST e Integração com Banco de Dados

Docente: Walter Viana

API REST

- Padrão de comunicação entre sistemas via HTTP, usando URLs e métodos como GET, POST, PUT e DELETE para acessar e manipular dados.
- Normalmente troca informações no formato JSON.

API REST

➤ HTTP

- **POST:** Envio de dados do cliente para o servidor, geralmente para criar um novo recurso.
- **GET:** Recuperação/exibição de dados do servidor para o cliente (navegador), sem alterar o estado do sistema.
- **PUT:** Envio de dados para o servidor com o objetivo de atualizar um recurso existente.
- **DELETE:** Requisição para remover um recurso existente no servidor.

API REST

➤ 🔦 API, REST e JSON

- **API:** permite a comunicação entre sistemas
- **REST:** padrão de APIs baseado em URLs e métodos HTTP
- **JSON:** formato leve para troca de dados

```
curl -X 'POST' \  
  'http://localhost:3000/api/v1/auth/login' \  
  -H 'accept: application/json' \  
  -H 'Content-Type: application/json' \  
  -d '{  
    "email": "jonas1@gmail.com",  
    "password": "123456"  
  }'
```

API REST

➤ 💡 API, REST e JSON

- **API:** permite a comunicação entre sistemas
- **API do exemplo:**
<http://localhost:3000/api/v1>
- Tudo que vem depois disso são funcionalidades da API.

```
curl -X 'POST' \  
  'http://localhost:3000/api/v1/auth/login' \  
  -H 'accept: application/json' \  
  -H 'Content-Type: application/json' \  
  -d '{  
    "email": "jonas1@gmail.com",  
    "password": "123456"  
  }'
```

API REST

➤ 🔦 API, REST e JSON

- **REST:** padrão de APIs baseado em URLs e métodos HTTP
- **REST do exemplo:** Método HTTP (POST)
- Isso caracteriza uma **API REST**.

```
curl -X 'POST' \
  'http://localhost:3000/api/v1/auth/login' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "email": "jonas1@gmail.com",
    "password": "123456"
  }'
```

API REST

➤ 💡 API, REST e JSON

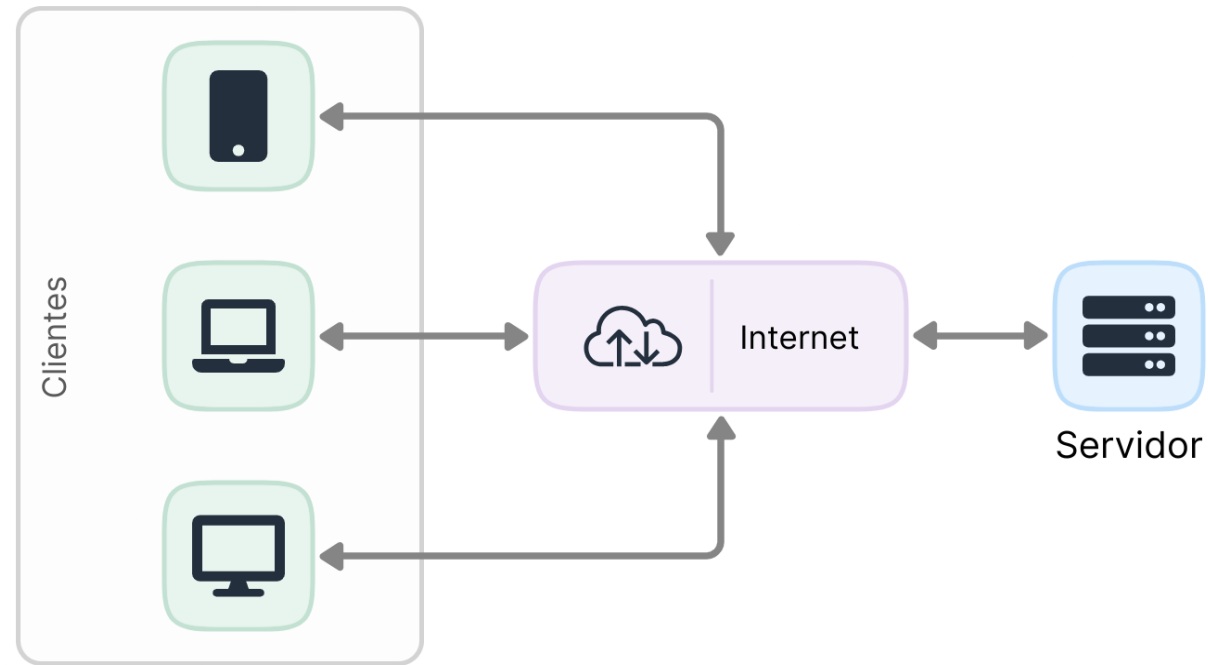
➤ **JSON:** formato leve para troca de dados

➤ **JSON do exemplo:**

```
{  
  "email": "jonas1@gmail.com",  
  "password": "123456"  
}
```

```
curl -X 'POST' \  
  'http://localhost:3000/api/v1/auth/login' \  
  -H 'accept: application/json' \  
  -H 'Content-Type: application/json' \  
  -d '{  
    "email": "jonas1@gmail.com",  
    "password": "123456"  
  }'
```

Arquitetura Genérica (Cliente-Servidor)



Arquitetura Genérica (Cliente-Servidor)

➤ Sugestões para implementar o servidor:

➤ Próprio



+



ou



+



Arquitetura Genérica (Cliente-Servidor)

➤ Sugestões para implementar o servidor:

➤ Externo

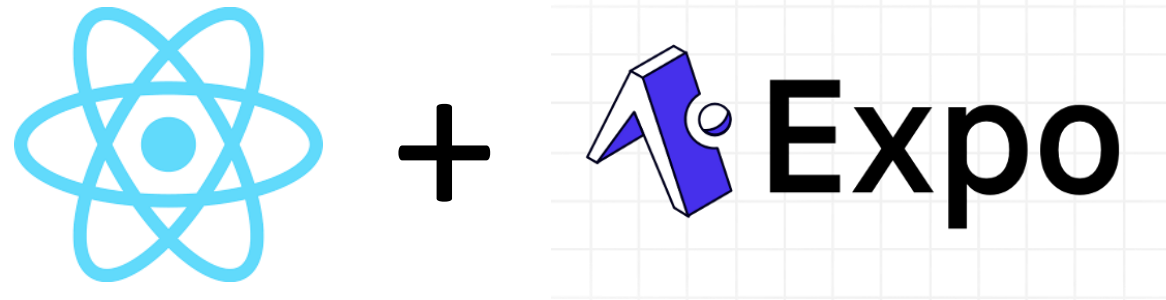


ou



Arquitetura Genérica (Cliente-Servidor)

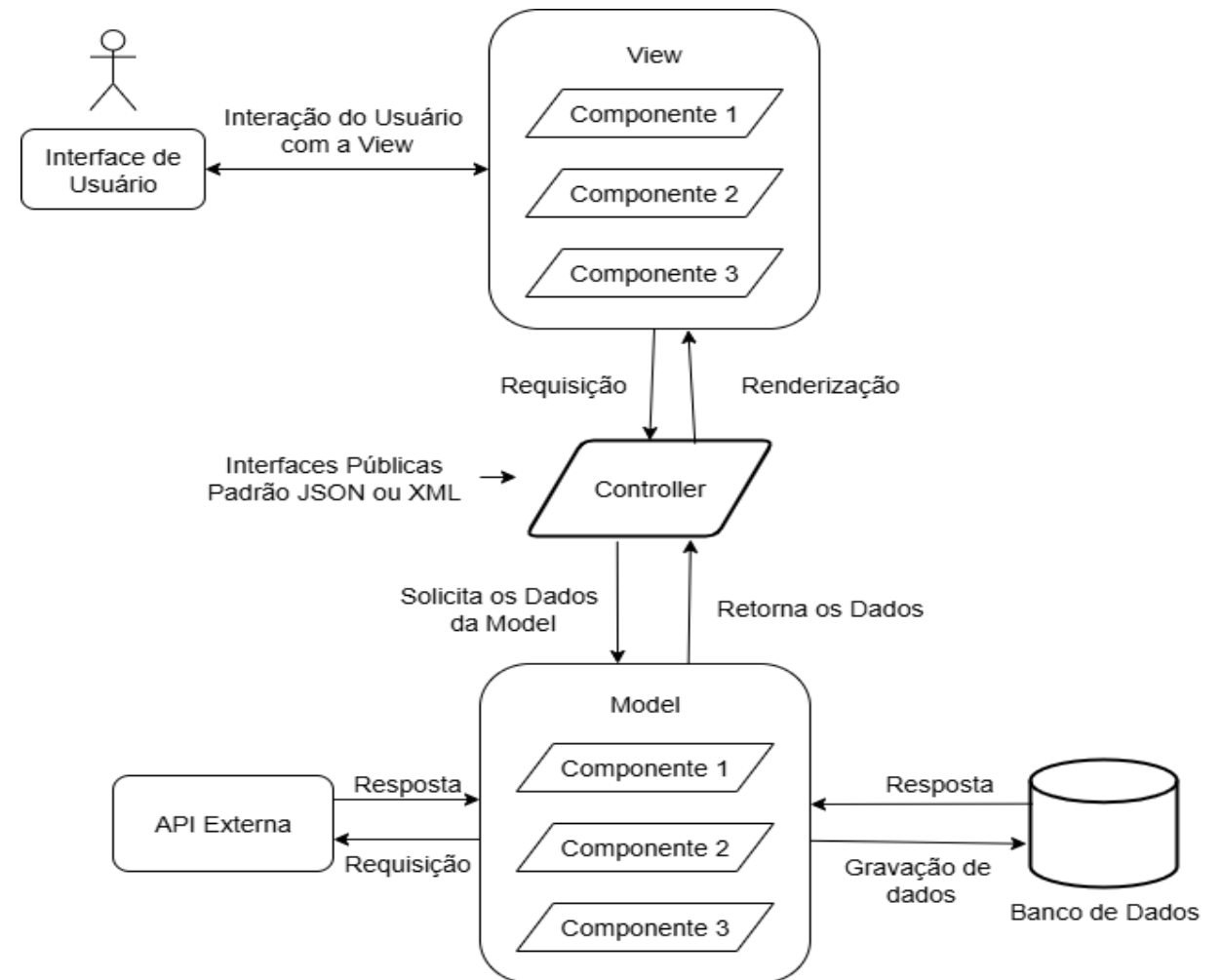
- Sugestões para implementar o cliente:



- Outras opções:
 - React Native Puro
 - Java/Kotlin no Android Studio -> Aqui você faz tanto Cliente quanto Servidor

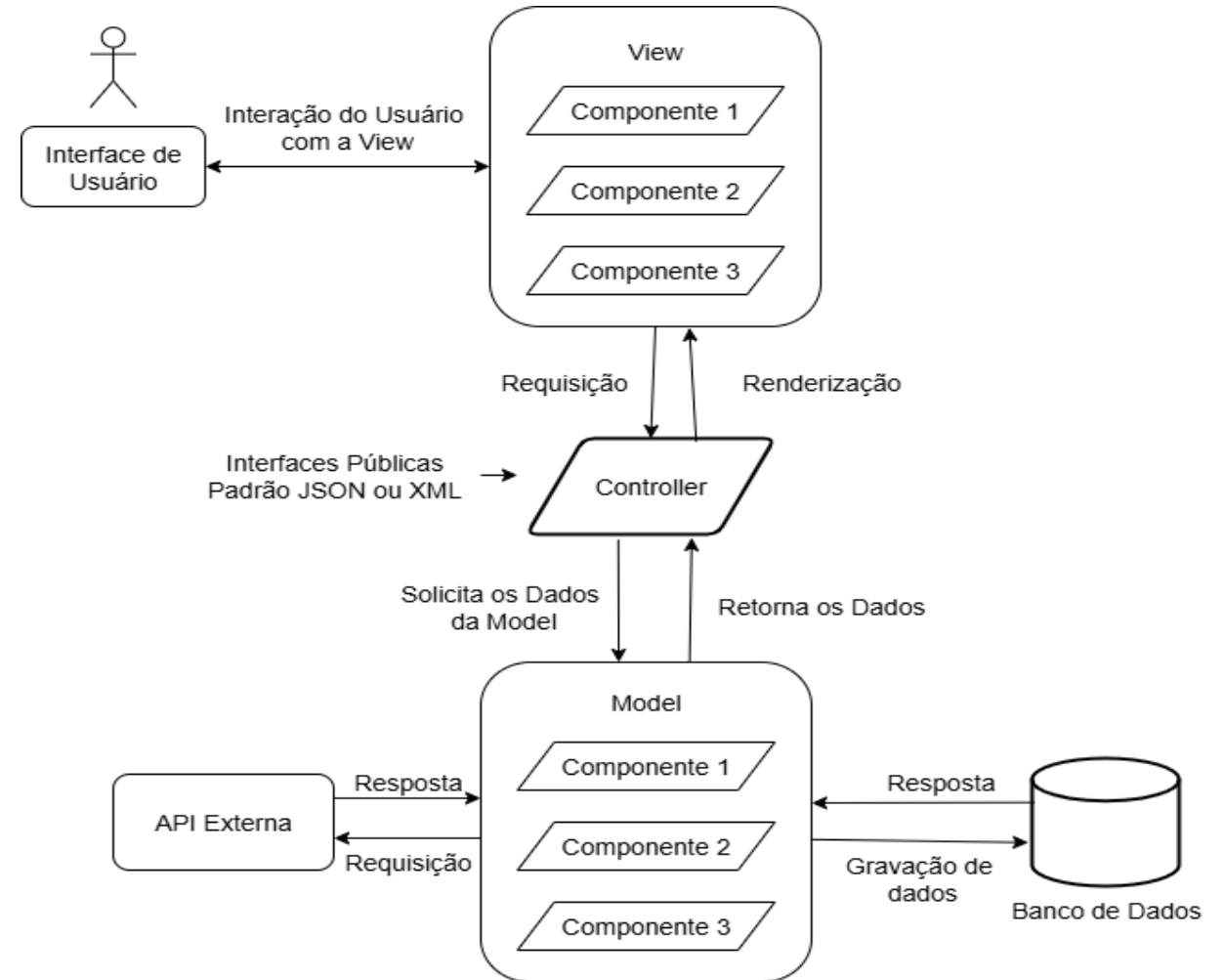
Arquitetura MVC

- **Model:** Responsável pelos dados e regras de negócio (acesso ao banco, validações).
- **View:** Interface com o usuário (telas, formulários, respostas visuais).
- **Controller:** Recebe as requisições, processa a lógica e conecta View e Model.



Arquitetura MVC

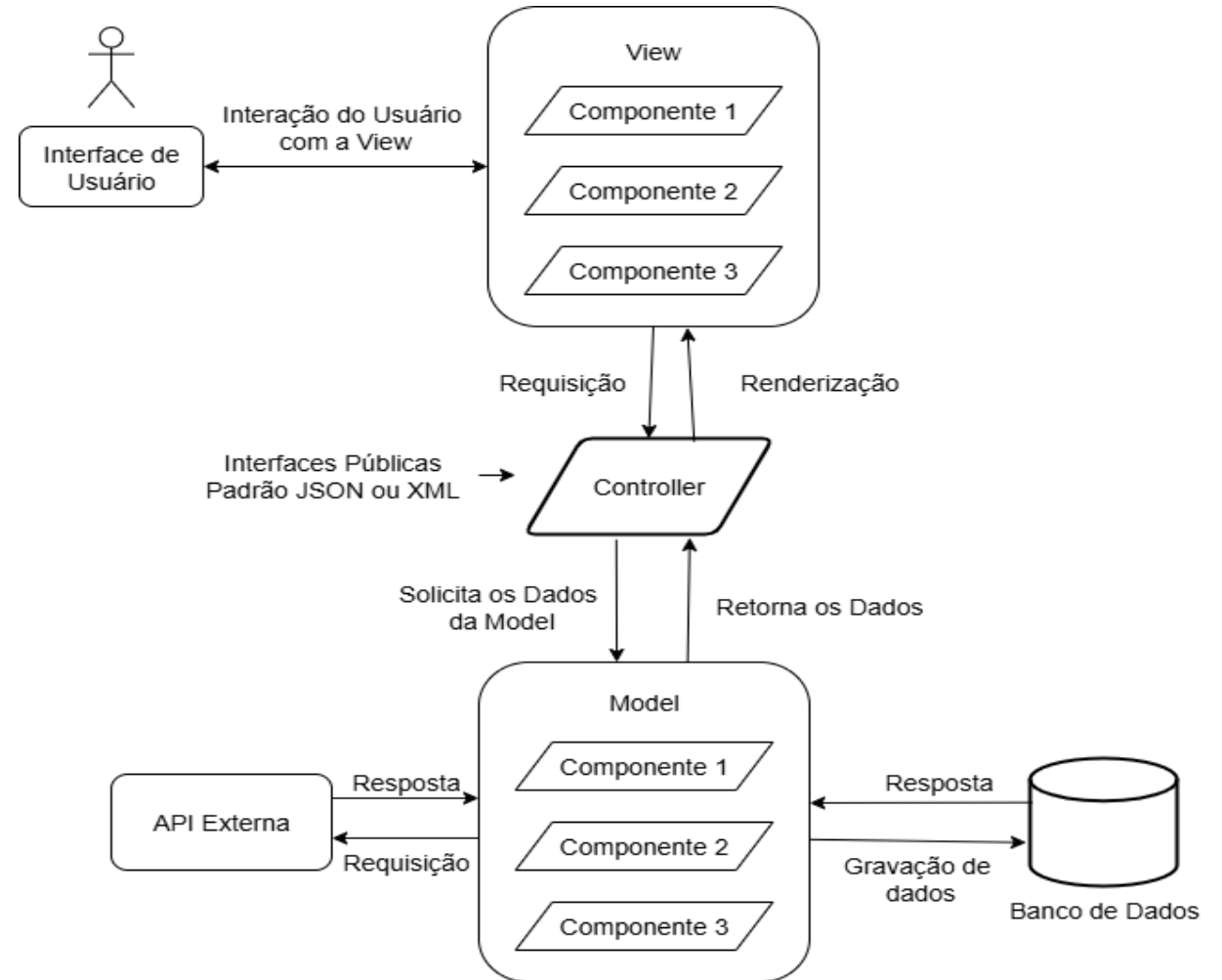
- **O Model e o Controller são implementados no backend**
- **O View é implementado no frontend**



Arquitetura MVC

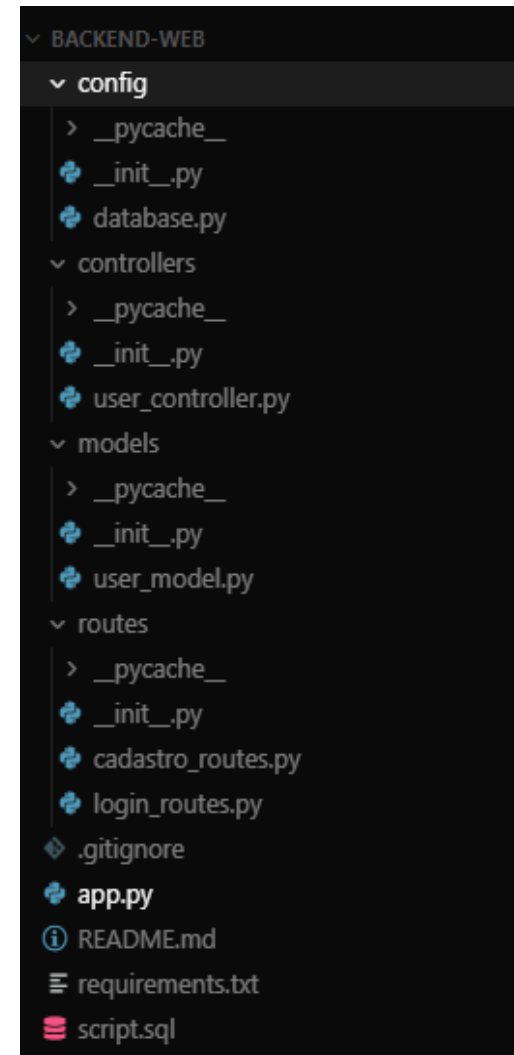
- Conseguimos implementar **Model** e **Controller** com algumas tecnologias e frameworks de backend:

- Linguagem Python com seus frameworks: Flask, FastAPI, Django.
- Linguagem Javascript com seus frameworks: Next.js, Node.js, ...
- Etc ...



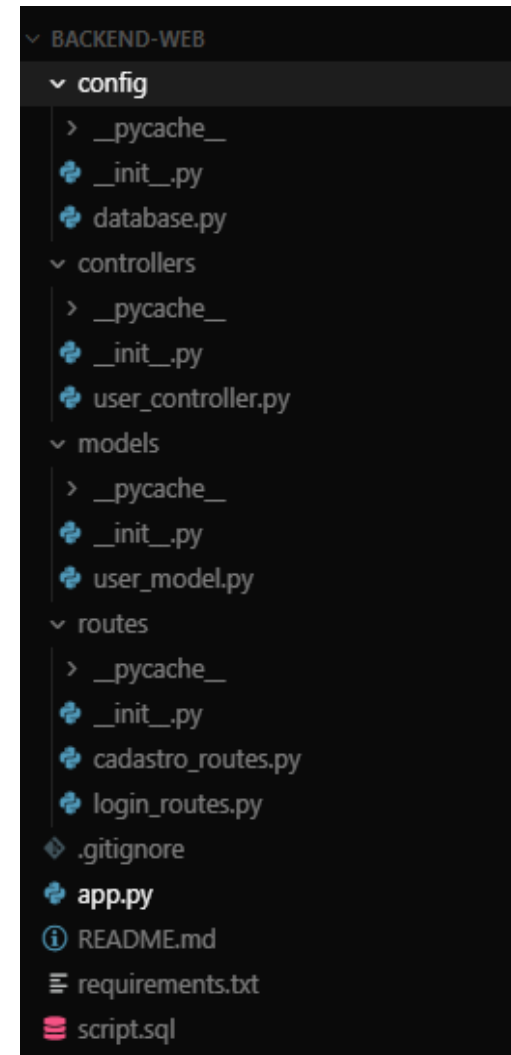
Arquitetura MVC

- Estrutura em python com Flask:



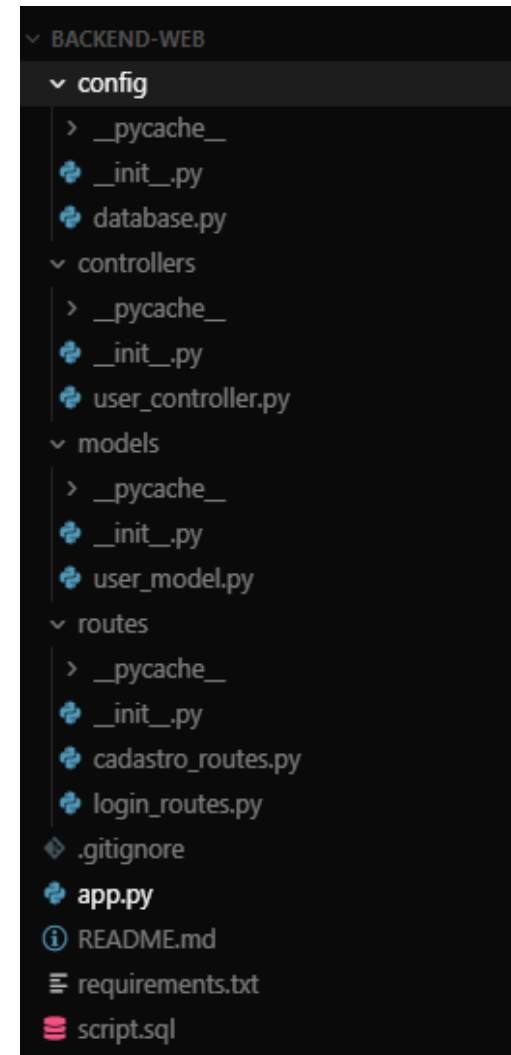
Arquitetura MVC

- **Model:** Responsável pelos dados e regras de negócio (ORM, acesso ao banco, validações).
- **Controllers:** Processam as requisições, aplicam a lógica e retornam respostas.



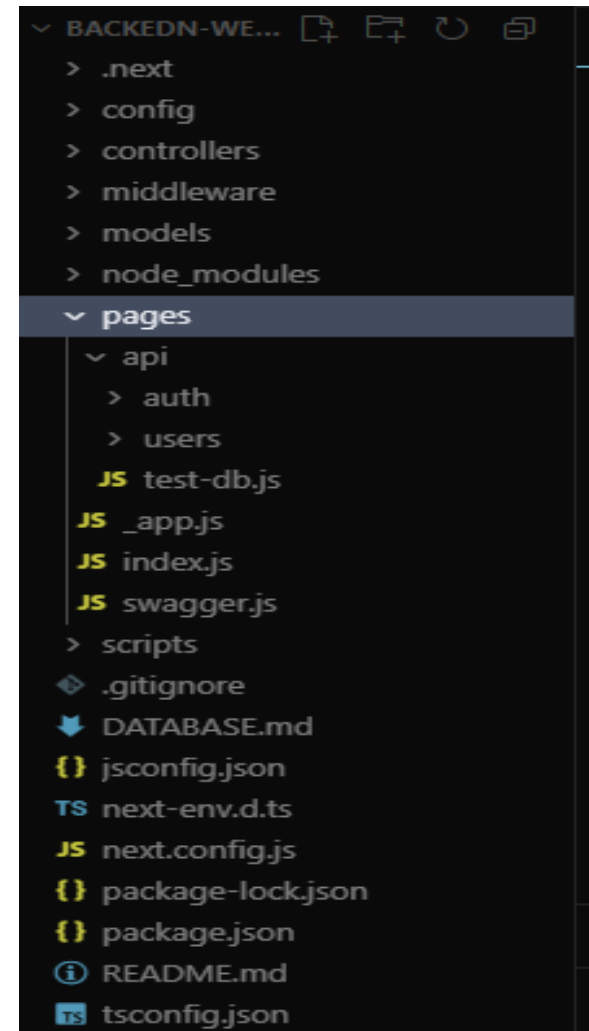
Arquitetura MVC

- **Routes:** Definem as URLs/endpoints e direcionam para os controllers.
- **Config:** Configurações da aplicação (banco de dados, variáveis de ambiente, chaves, portas, etc.).



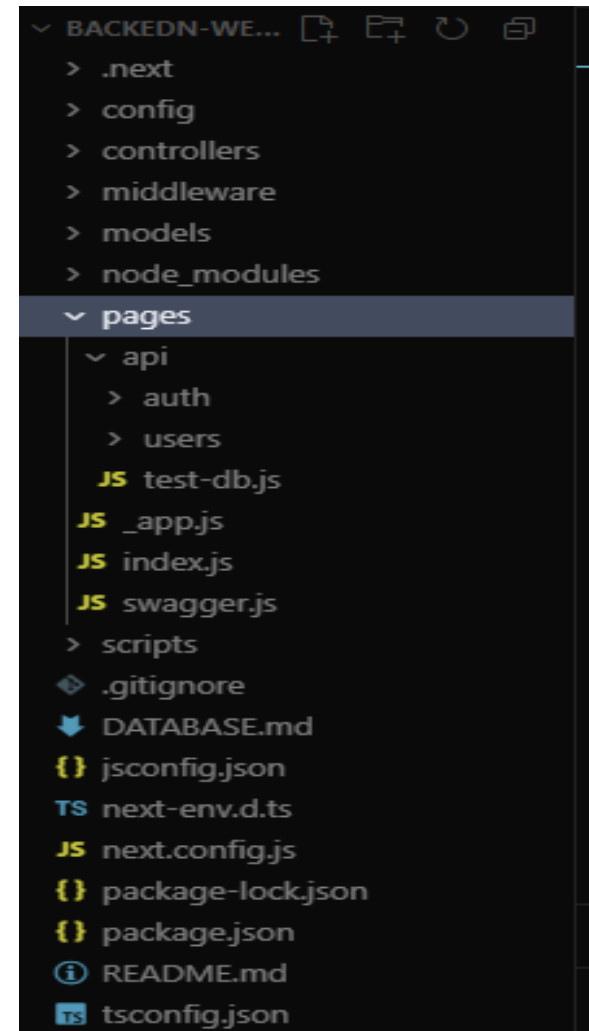
Arquitetura MVC

- Estrutura em Javascript com Next:



Arquitetura MVC

- Estrutura em Javascript com Next:
- Pages tem a mesma função que o routes no flask.

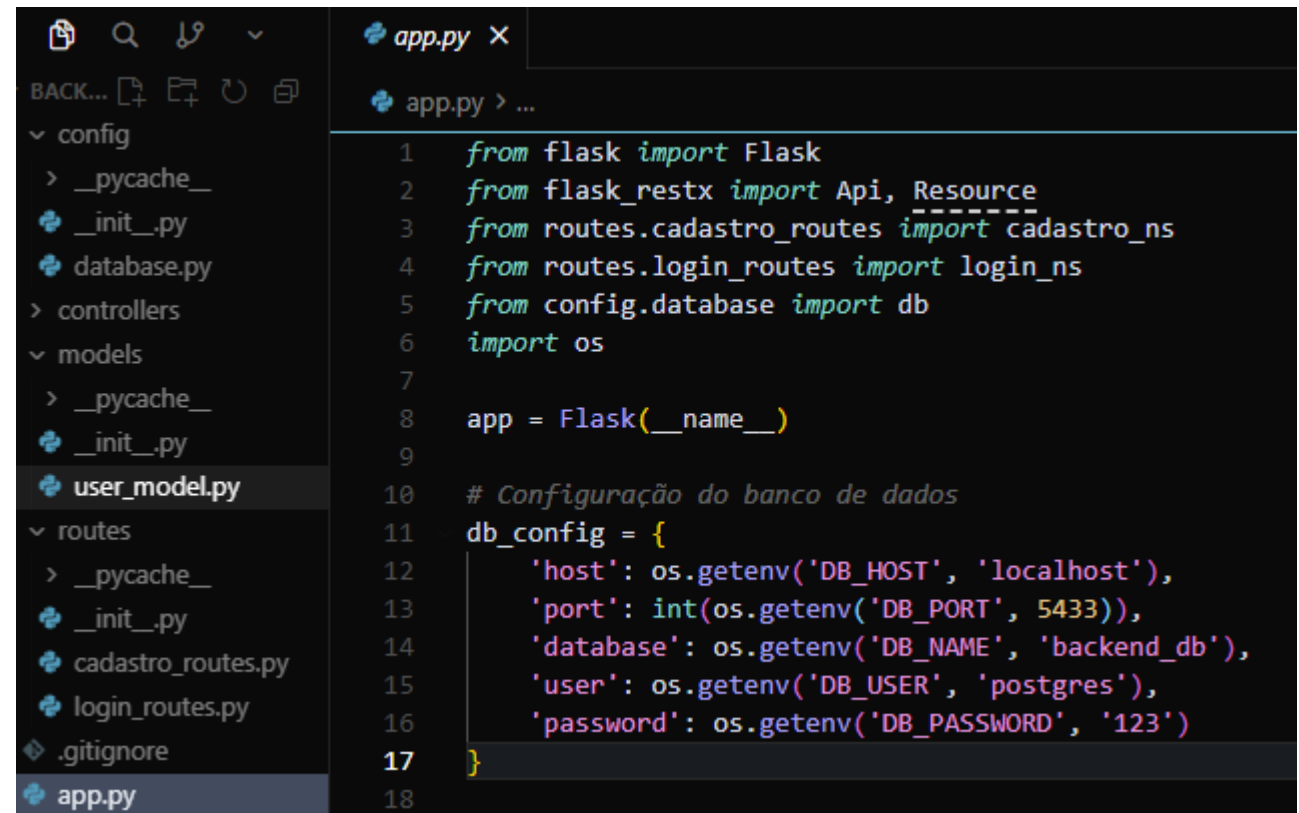


Arquitetura MVC

- Estrutura do framework ≠ Arquitetura do sistema
- Frameworks como Flask e Next organizam pastas e arquivos de formas diferentes. Mesmo assim, ainda é possível aplicar padrões arquiteturais como MVC.
- Isso porque, independentemente da estrutura:
 - existe interface (View)
 - existem rotas/controladores (Controller)
 - existe lógica de negócio e acesso a dados (Model/Service)
- Ou seja, a organização muda, mas o conceito arquitetural permanece.

Arquitetura MVC

- Integração com o banco de dados
- Flask:



```
app.py X
app.py > ...

1  from flask import Flask
2  from flask_restx import Api, Resource
3  from routes.cadastro_routes import cadastro_ns
4  from routes.login_routes import login_ns
5  from config.database import db
6  import os
7
8  app = Flask(__name__)
9
10 # Configuração do banco de dados
11 db_config = {
12     'host': os.getenv('DB_HOST', 'localhost'),
13     'port': int(os.getenv('DB_PORT', 5433)),
14     'database': os.getenv('DB_NAME', 'backend_db'),
15     'user': os.getenv('DB_USER', 'postgres'),
16     'password': os.getenv('DB_PASSWORD', '123')
17 }
18
```

Arquitetura MVC

- Integração com o banco de dados

- Next.js:



```
4 // Configuração do banco de dados PostgreSQL
5 const pool = new Pool({
6   host: process.env.DB_HOST || 'localhost',
7   port: parseInt(process.env.DB_PORT || '5433'),
8   database: process.env.DB_NAME || 'backend_db',
9   user: process.env.DB_USER || 'postgres',
10  password: process.env.DB_PASSWORD || '123',
11  max: 20, // máximo de clientes no pool
12  idleTimeoutMillis: 30000,
13  connectionTimeoutMillis: 2000,
14 });
15
```

Documentação da API

- **Para que serve:** Facilitar o entendimento, uso, testes e integração entre sistemas.
- Ferramentas:
 - Postman → testar e organizar requisições
 - Insomnia → similar ao Postman
 - Swagger (OpenAPI) → documentar e visualizar endpoints da API
 - Etc ...

API Backend ^{1.0}

[Base URL: /api]
[/api/swagger.json](#)

API com cadastro e login

cadastro Operações de cadastro de usuários ^

GET /cadastro Lista todos os usuários cadastrados

POST /cadastro Cria um novo usuário

GET /cadastro/{user_id} Busca um usuário por ID

PUT /cadastro/{user_id} Atualiza um usuário

DELETE /cadastro/{user_id} Deleta um usuário

login Operações de login ^

POST /login Realiza login do usuário

👉 Swagger é o mais popular e amplamente usado no mercado.

Documentação da API

- Roda na porta do backend (ex.: localhost:3000, localhost:3001, ...).
- Permite testar operações de CRUD diretamente na API, sem depender do frontend.

API Backend ^{1.0}

[Base URL: /api]
[/api/swagger.json](#)

API com cadastro e login

cadastro Operações de cadastro de usuários ^

GET	/cadastro	Lista todos os usuários cadastrados	▼
POST	/cadastro	Cria um novo usuário	▼
GET	/cadastro/{user_id}	Busca um usuário por ID	▼
PUT	/cadastro/{user_id}	Atualiza um usuário	▼
DELETE	/cadastro/{user_id}	Deleta um usuário	▼

login Operações de login ^

POST	/login	Realiza login do usuário	▼
------	--------	--------------------------	---

Status HTTP: respostas de sucesso e erro em APIs

➤ **200 – OK** -> Requisição realizada com sucesso.

➤ **400 – Bad Request** -> Erro na requisição (dados inválidos ou incompletos).

Ex.: enviar cadastro sem email ou senha.

➤ **401 – Unauthorized** -> Usuário não autenticado.

Ex.: acessar rota protegida sem login/token.

Status HTTP: respostas de sucesso e erro em APIs

➤ **404 – Not Found** -> Recurso ou rota não encontrada.

Ex.: acessar /users/99 quando esse usuário não existe.

➤ **405 – Method Not Allowed** -> Método HTTP não permitido para a rota.

Ex.: usar DELETE em uma rota que aceita apenas GET.

➤ **409 – Conflict** -> Conflito de dados (registro já existente).

Ex.: tentar cadastrar um email que já está no banco.

Ferramentas para consumo de APIs (Fetch Vs Axios)

- **Fetch** e **Axios** são ferramentas usadas no frontend para fazer requisições HTTP a APIs (buscar, enviar, atualizar ou remover dados).
- O que são / para que servem
 - Fetch: API nativa do JavaScript para requisições HTTP no navegador.
 - Axios: biblioteca externa que faz requisições HTTP com mais recursos prontos.
- Servem para consumir APIs: GET, POST, PUT, DELETE, etc.

Noções de Segurança em Servidores

- **JWT (JSON Web Token)**
Usado para autenticação e autorização de usuários via tokens.
- **CORS (Cross-Origin Resource Sharing)**
Controla quais origens podem acessar a API, evitando acessos não autorizados.
- **SQL Injection**
Ataque que insere comandos SQL maliciosos em formulários ou requisições para acessar ou alterar dados.
- **Criptografia**
Técnica de proteção de dados sensíveis por meio de codificação

Noções de Segurança em Servidores

➤ JWT (JSON Web Token)

Request URL

http://127.0.0.1:3000/api/login

Server response

Code

Details

200

Response body

```
{
  "message": "Login realizado com sucesso",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoyLCJlbWFpbCI6ImpvbmFzQGdtYWlsLmNvbSI6ImV4cCI6MTc3MDczMDk1MCwiaWF0IjoxNzcwNjQ0NTUwfQ.GJzLnqZaGqX0W4w1en0gcyXy1nLsG1W1PndFWtaxbhzY",
  "user": {
    "id": 2,
    "nome": "jonas",
    "email": "jonas@gmail.com"
  }
}
```

 Download

Response headers

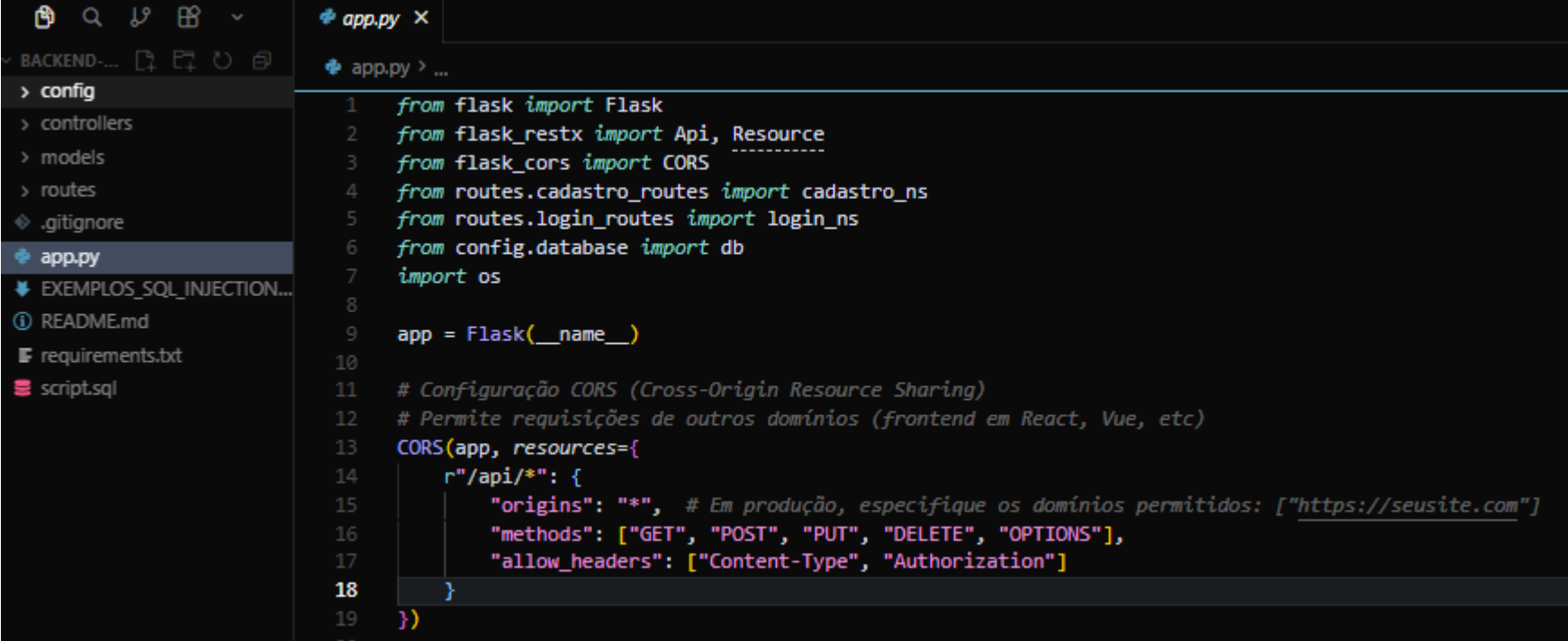
Noções de Segurança em Servidores

➤ CORS (Cross-Origin Resource Sharing)



Noções de Segurança em Servidores

➤ CORS (Cross-Origin Resource Sharing)



The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with folders like config, controllers, models, routes, and files like .gitignore, app.py, README.md, requirements.txt, and script.sql. The code editor shows the content of app.py, which includes imports for Flask, flask_restx, flask_cors, and routes, followed by the initialization of the Flask app and the CORS configuration.

```
1 from flask import Flask
2 from flask_restx import Api, Resource
3 from flask_cors import CORS
4 from routes.cadastro_routes import cadastro_ns
5 from routes.login_routes import login_ns
6 from config.database import db
7 import os
8
9 app = Flask(__name__)
10
11 # Configuração CORS (Cross-Origin Resource Sharing)
12 # Permite requisições de outros domínios (frontend em React, Vue, etc)
13 CORS(app, resources={
14     r"/api/*": {
15         "origins": "*", # Em produção, especifique os domínios permitidos: ["https://seusite.com"]
16         "methods": ["GET", "POST", "PUT", "DELETE", "OPTIONS"],
17         "allow_headers": ["Content-Type", "Authorization"]
18     }
19 })
```

Noções de Segurança em Servidores

➤ SQL Injection

Ex.:

- Query vulnerável (concatenação de string): **SELECT * FROM users WHERE email = '' + email + '' AND password = '' + password + ''**;

➡ Permite inserir SQL malicioso.

- Exemplo de SQL malicioso: ' **OR '1'='1**
- Isso pode autenticar sem senha válida.

Noções de Segurança em Servidores

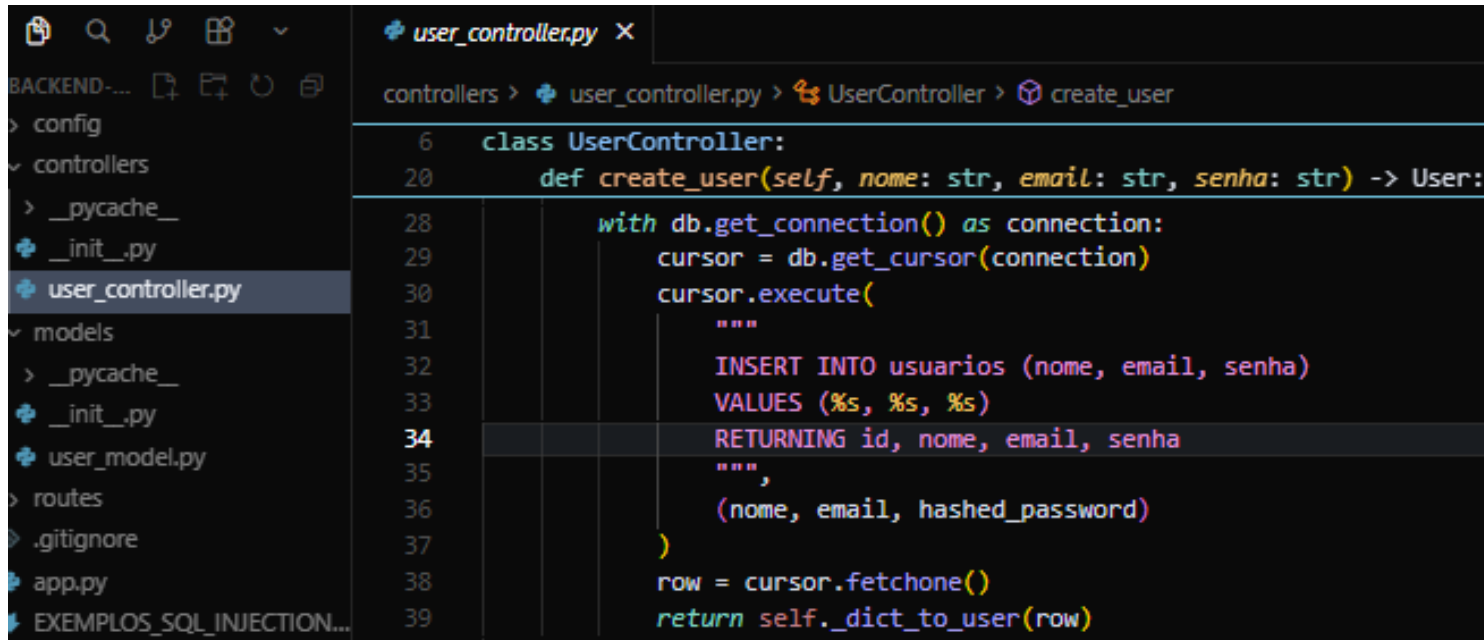
- Query segura (parametrizada): **SELECT * FROM users WHERE email = ? AND password = ?;**
- ➡ Usa parâmetros preparados, evitando injeção de SQL.

Noções de Segurança em Servidores

- Se alguém enviar: **OR 1=1**
- O banco NÃO executa isso como lógica SQL. Ele interpreta literalmente como string: **email = "OR 1=1"**
- Ou seja:
 - vira apenas um valor de campo
 - não altera a estrutura da query
 - não “quebra” o WHERE
- Já na versão vulnerável (concatenação): **WHERE email = "" + email + ""** . O conteúdo entra direto no SQL e passa a fazer parte do comando.

Noções de Segurança em Servidores

➤ SQL Injection



```
user_controller.py X
controllers > user_controller.py > UserController > create_user

6 class UserController:
20     def create_user(self, nome: str, email: str, senha: str) -> User:
28         with db.get_connection() as connection:
29             cursor = db.get_cursor(connection)
30             cursor.execute(
31                 """
32                 INSERT INTO usuarios (nome, email, senha)
33                 VALUES (%s, %s, %s)
34                 RETURNING id, nome, email, senha
35                 """,
36                 (nome, email, hashed_password)
37             )
38             row = cursor.fetchone()
39             return self._dict_to_user(row)
```

Noções de Segurança em Servidores

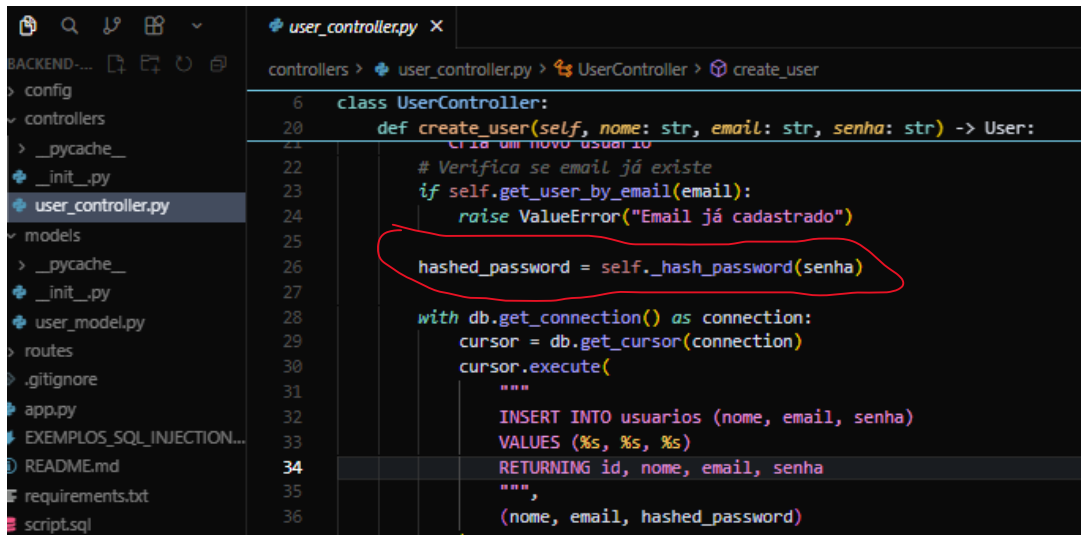
➤ **SQL Injection**

- **Resumo do Sql Injetion:** ? e %s separam código SQL de dados do usuário. O banco executa o SQL primeiro e só depois encaixa os valores, impedindo que comandos maliciosos sejam interpretados como instruções SQL.

Noções de Segurança em Servidores

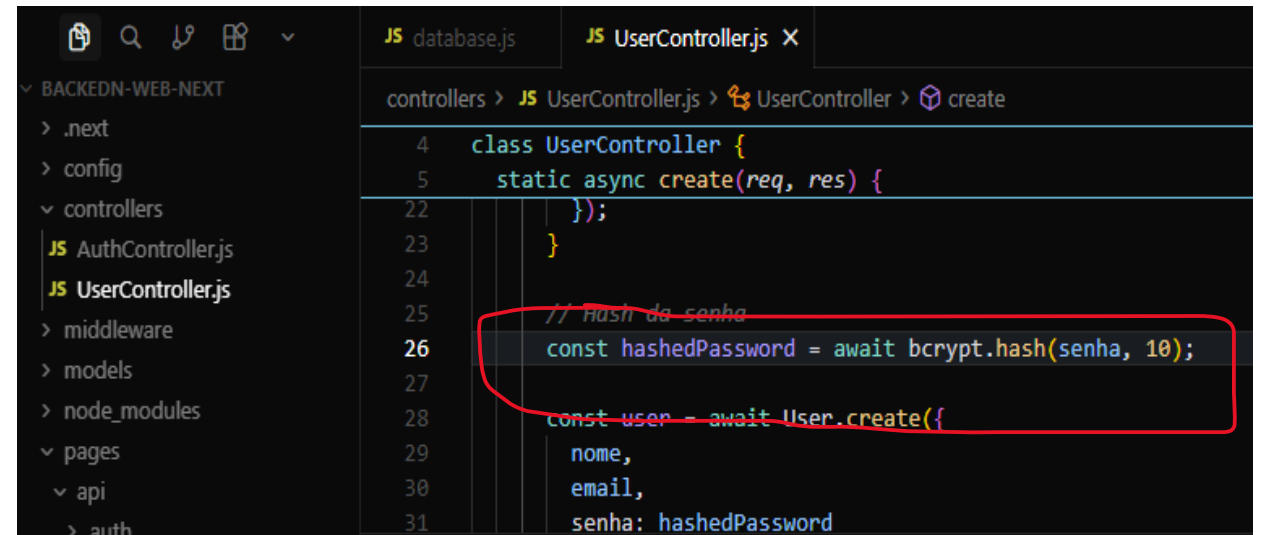
➤ Criptografia

Python



```
6 class UserController:
20     def create_user(self, nome: str, email: str, senha: str) -> User:
21         # Cria um novo usuário
22         # Verifica se email já existe
23         if self.get_user_by_email(email):
24             raise ValueError("Email já cadastrado")
25
26         hashed_password = self._hash_password(senha)
27
28         with db.get_connection() as connection:
29             cursor = db.get_cursor(connection)
30             cursor.execute(
31                 """
32                 INSERT INTO usuarios (nome, email, senha)
33                 VALUES (%s, %s, %s)
34                 RETURNING id, nome, email, senha
35                 """,
36                 (nome, email, hashed_password)
37             )
```

JavaScript



```
JS database.js JS UserController.js X
controllers > JS UserController.js > UserController > create
4 class UserController {
5     static async create(req, res) {
22     };
23 }
24
25 // Hash da senha
26 const hashedPassword = await bcrypt.hash(senha, 10);
27
28 const user = await User.create({
29     nome,
30     email,
31     senha: hashedPassword
32 });
```

Linguagem Pura Vs Com Frameworks

Python puro Vs Python com Flask, JavaScript Puro Vs JavaScript com Next

Puro	Com Frameworks
Organização manual	Estrutura pré-definida e padronizada
Mais código e configuração	Acelera o desenvolvimento com recursos prontos
Implementação do zero	Rotas e endpoints já estruturados
Depende totalmente da organização do desenvolvedor	Favorece boas práticas e organização
Precisa integrar tudo manualmente	Autenticação, middleware, templates, etc. já disponíveis

Referências

QUEIRÓS, Ricardo; PORTELA, Filipe. Introdução ao Desenvolvimento Moderno Para a Web. Do Front-End ao Back-End. Uma Visão Global. FCA. 1a edição. 2018.

MUELLER, John Paul; KINOSHITA, Lúcia. Segurança Para Desenvolvedores Web: Usando JavaScript, HTML e CSS. ISBN-10: 8575224840. ISBN-13: 978-8575224847. Editora: Novatec; Edição: 1a (21 de março de 2016).

PRESSMAN, R. S. Engenharia de Software: uma abordagem profissional. 7.ed. São Paulo: McGraw-Hill, 2011.

SOMMERVILLE, I. Engenharia de Software. 9.ed. São Paulo: Pearson Prentice Hall, 2011.